

Zadanie 17

Natalia Włódyka

Styczeń 06, 2026

1 Polecenie zadania

Celem zadania jest znalezienie minimum poniższej funkcji:

$$f(x) = \frac{1}{4}x^4 - \frac{1}{2}x^2 - \frac{1}{16}x$$

Za pomocą metody złotego podziału oraz metody Brenta

2 Metoda złotego przedziału

Minimum funkcji wyliczamy za pomocą iteracji zawężającej przedział $[a, c]$, która go zawiera. W każdej iteracji wyliczamy punkt d :

$$d = \left\{ \begin{array}{ll} a + w|b - a|, & \text{dla } |b - a| > |c - b| \\ b + w|c - a|, & \text{dla } |b - a| \leq |c - b| \end{array} \right\} \quad (1)$$

Gdzie:

$$w = \frac{3 - \sqrt{5}}{2}$$

Następnie zawężamy odpowiednio przedział: Jeśli $f(d) < f(b)$:

$$\left\{ \begin{array}{ll} c = b, & b = d \quad \text{dla } d < b \\ a = b, & b = d \quad \text{dla } d \geq b \end{array} \right\}$$

W przeciwnym wypadku:

$$\left\{ \begin{array}{ll} a = d & \text{dla } d < b \\ c = d & \text{dla } d \geq b \end{array} \right\}$$

Iteracje kończymy, gdy spełnione jest poniższe równanie:

$$|c - a| < \varepsilon(|b| + |d|)$$

3 Metoda Brenta

Minimum paraboliczne, dla trzech punktów a, b i c , wyliczamy ze wzoru:

$$d = \frac{1}{2} * \frac{a^2(f_c - f_b) + b^2(f_a - f_c) + c^2(f_b - f_a)}{a(f_c - f_b) + b(f_a - f_c) + c(f_b - f_a)}$$

Jeśli punkty d nie należy do przedziału (a, c) , wykonujemy wtedy krok bisekcji:

$$d = \frac{a + c}{2}$$

Iteracje kończymy, gdy spełnione jest poniższe równanie:

$$|c - a| < \varepsilon(|b| + |d|)$$

4 Kod w Pythonie

```
1 import numpy as np
2
3 numb = np.float64
4
5 def f_x(x: numb) -> numb:
6     coefficients = np.array([0.0, -1/16, -0.5, 0.0, 0.25], dtype=
7         numb)
8     value = 0.0
9
10    for i in range(0, len(coefficients)):
11        value += coefficients[i] * (x ** i)
12
13    return value
14
15 def bracket_minimum(x: numb) -> tuple[numb, numb, numb]:
16     a = x
17     h = 0.1
18
19     f_a = f_x(a)
20
21     b = a + h
22     f_b = f_x(b)
23
24     if f_b > f_a:
25         h = -h
26         b = a + h
27         f_b = f_x(b)
28         if f_b > f_a:
29             raise RuntimeError("Brak kierunku spadku")
30
31     for _ in range(100):
32         c = b + h
33         f_c = f_x(c)
34
35         if f_c > f_b:
36             return a, b, c
```

```

36
37     a, f_a = b, f_b
38     b, f_b = c, f_c
39     h *= 2
40
41     raise RuntimeError("Funkcja monotoniczna")
42
43 def gold_ratio_method(x:numb) -> numb:
44     a, b, c = bracket_minimum(x)
45     print(f"a = {a}, b = {b}, c = {c}")
46     w = (3 - np.sqrt(5)) / 2
47     tolerance : float = 1e-6
48     iteration : int = 100
49
50     for i in range(1, iteration):
51
52         if abs(b - a) > abs(c - b):
53             d = a + w * abs(b - a)
54         else:
55             d = b + w * abs(c - b)
56
57         if abs(c - a) < tolerance * (abs(b) + abs(d)):
58             print(f"Minimum wynosi: = {b}, a znaleziono je w {i}
iteracji")
59             break
60
61         f_b, f_d = f_x(b), f_x(d)
62
63         if f_d < f_b:
64             if d < b:
65                 c, b = b, d
66             else:
67                 a, b = b, d
68         else:
69             if d < b:
70                 a = d
71             else:
72                 c = d
73
74     return b
75
76 def brent_method(x: numb) -> numb:
77     a, b, c = bracket_minimum(x)
78     tolerance : float = 1e-6
79     iteration : int = 100
80     prev_interval = abs(c - a)
81     prev2_interval = prev_interval
82
83     for i in range(1, iteration):
84         old_b = b
85         f_a, f_b, f_c = f_x(a), f_x(b), f_x(c)
86         acceptable = False
87
88         denominator = a * (f_c - f_b) + b * (f_a - f_c) + c * (f_b -
f_a)
89
90         if abs(denominator) > 1e-14:

```

```

91     numerator = (a**2) *(f_c - f_b) + (b**2) * (f_a - f_c)
92     + (c**2) * (f_b - f_a)
93     d = (1/2) * (numerator/denominator)
94
95     if a < d < c:
96         new_interval = max(abs(d - a), (c - d))
97         if new_interval <= 0.5 * prev2_interval:
98             acceptable = True
99
100     if not acceptable:
101         d = (c + a) / 2.0
102
103     f_d = f_x(d)
104
105     if f_d < f_b:
106         if d < b:
107             c, b = b, d
108         else:
109             a, b = b, d
110
111     else:
112         if d < b:
113             a = d
114         else:
115             c = d
116
117     prev2_interval = prev_interval
118     prev_interval = abs(c - a)
119
120     if abs(c - a) < tolerance * (abs(old_b) + abs(d)):
121         print(f"Minimum wynosi: = {b}, a znalezione je w {i}
122         iteracji")
123         break
124
125     return b
126
127 def main():
128     brent_minimum = brent_method(0.0)
129     gold_ratio_minimum = gold_ratio_method(0.0)
130
131     brent_minimum = brent_method(-3.0)
132     gold_ratio_minimum = gold_ratio_method(-3.0)
133
134 if __name__ == "__main__":
135     main()

```

5 Minimum wielomianu

Dla $a = 0.4$, $b = 0.8$, $c = 1.6$:

Metoda Brenta: $x_{min} = 1.0298959$, $iteracje = 28$

Metoda złotego środka: $x_{min} = 1.0298959$, $iteracje = 34$

Dla $a \approx -1.6$ $b \approx -1.2$, $c \approx -0.4$:

Metoda Brenta: $x_{min} = -0.9671490$, $iteracje = 15$
Metoda złotego środka: $x_{min} = -0.9671489$, $iteracje = 39$

Jak widać powyżej, zbieżność metody Brenta jest znacząco szybsza od złotego przedziału.